Teoría de Algoritmos
(TB024/75.29) Curso Buchwald - Genender

Trabajo Práctico 2
Programación Dinámica para el Reino de la Tierra

11 de octubre de 2025

Facundo Camilo Stifman
112541

Juan Ignacio Moore
112479

Avril Victoria Morfeo Zerbi
112563

Martin Ariel Kupa
112560

Índice

1. Análisis algorítmico **3**
1.1. Enunciado del problema . . . 3
1.2. Algoritmo propuesto . . . 3
1.3. Ejemplos de ejecucion . . . 4
1.3.1. Ejemplo 1: Recarga rápida . . . 4
1.3.2. Ejemplo 2: Recarga lenta . . . 5
1.3.3. Ejemplo 3: Oleadas con ceros intermedios . . . 5
1.3.4. Ejemplo 4: Recarga total (siempre alcanza) . . . 5
1.3.5. Conclusiones del seguimiento . . . 6

2. Demostración **6**
2.1. Caso inductivo . . . 6
2.2. Caso base . . . 6
2.3. Generalidad de la conclusión . . . 6

3. Mediciones **6**
3.1. Conjunto balanceado . . . 7
3.1.1. Error del ajuste . . . 7
3.2. Conjunto mayor cantidad de soldados (tilt_x) . . . 8
3.2.1. Error del ajuste . . . 8
3.3. Conjunto mayor capacidad de recarga (tilt_e) . . . 9
3.3.1. Error del ajuste . . . 9
3.4. Análisis de resultados . . . 10
3.4.1. Comparación entre algoritmos . . . 11
3.4.2. Impacto de la variabilidad en los datos . . . 11
3.5. Conclusión . . . 11

A. Resultados sobre sets de la catedra **12**

B. Resultados de las mediciones **12**

C. Código de generación de datos **13**

1. Análisis algorítmico

1.1. Enunciado del problema

Dada una función $f : \mathbb{N} \to \mathbb{N}_0 \backslash f(j)$ monótona creciente, que indica la cantidad máxima de enemigos que se pueden eliminar si se cargó energía durante j minutos, y una lista X de n números naturales:

$$X = (x_1, x_2, \ldots, x_n), \quad x_i \in \mathbb{N}_0$$

donde x_i es la cantidad de enemigos que llegan en el minuto i.

Sea $A = (i_1, i_2, \ldots, i_m)$ la secuencia de minutos en los que se decide atacar, con $1 \leq i_1 < i_2 < \ldots < i_m \leq n$, donde m es la cantidad de elementos en A (el número de ataques realizados).

La cantidad de enemigos eliminados en el k-ésimo ataque $i_k \in A$ depende de cuántos minutos pasaron desde el último ataque. Específicamente, si el ataque anterior fue en el minuto i_{k-1}, entonces en el minuto i_k se eliminan $\text{mín}(x_{i_k}, f(i_k - i_{k-1}))$ enemigos, donde por convención para el primer ataque se toma $i_0 = 0$.

La cantidad total de enemigos eliminados para una secuencia de ataques A es:

$$g(A) = \sum_{k=1}^{m} \text{mín}(x_{i_k}, f(i_k - i_{k-1}))$$

El objetivo es encontrar una secuencia óptima de ataques A^* que maximice $g(A)$:

$$A^* \in \arg\max_{A} g(A)$$

1.2. Algoritmo propuesto

Para encontrar la secuencia de ataques óptimos proponemos un algoritmo de programación dinámica basado en la siguiente ecuación de recurrencia. Sea $OPT(j)$ la cantidad máxima de enemigos derrotados considerando los primeros j minutos, se tiene que:

$$OPT(j) = \max_{i<j} (OPT(i) + \text{mín}(f(j-i), x_j))$$

Su implementación en python en la siguiente:

```python
def crear_opt(enemigos: list[int], calcular_muertos: list[int]):
  M = [0] * (len(enemigos) + 1)
  prev = [0] * (len(enemigos) + 1)
  for ultima_oleada in range(1, len(enemigos) + 1):
    maximo = min(calcular_muertos[ultima_oleada - 1], enemigos[ultima_oleada - 1])
    prev[ultima_oleada] = 0
    for ultimo_ataque in range(1, ultima_oleada):
      muertos = M[ultimo_ataque] + min(calcular_muertos[(ultima_oleada - ultimo_ataque) - 1], enemigos[ultima_oleada - 1])
      if muertos > maximo:
        maximo = muertos
        prev[ultima_oleada] = ultimo_ataque
    M[ultima_oleada] = maximo
  return M, prev

def reconstruir(enemigos: list[int], calcular_muertos: list[int]):
  M, prev = crear_opt(enemigos, calcular_muertos)

  oleada_actual = len(enemigos)
  acciones = []

  while oleada_actual > 0:
```

```
    ultimo_ataque = prev[oleada_actual]

    num_cargas = oleada_actual - ultimo_ataque - 1
    if num_cargas < 0:
      num_cargas = 0

    acciones.append(ATACAR)
    for _ in range(num_cargas):
      acciones.append(CARGAR)

    oleada_actual = ultimo_ataque

  acciones.reverse()
  return acciones, M[len(enemigos)]
```

El algoritmo construye la tabla de PD, lo que requiere trabajo lineal. Luego se recorre cada oleada, comparando con todas las oleadas anteriores en busca del máximo. La comparación es una operación constante, por lo cual el coste temporal esta dado por:

$$\sum_{i=1}^{n}\sum_{j=1}^{i} 1 = \frac{n(n+1)}{2}$$

Note que a medida que calculamos los valores de OPT a su vez guardamos el minuto del último ataque, esto permite una reconstrucción en tiempo lineal. El proceso de reconstrucción parte del n-ésimo valor de la lista de PD y:

1. marca que se debe atacar en ese minuto;
2. busca el minuto del ataque anterior (trabajo constante si la almacenamos previamente);
3. marca que se debe cargar en los minutos intermedios hasta llegar al ataque anterior;
4. repite desde el primer paso hasta agotar los minutos.

Al recorrer todos los minutos el trabajo resulta lineal. Finalmente, la complejidad temporal es:

$$O\left(\frac{n(n+1)}{2} + 2n\right) = O(n^2)$$

1.3. Ejemplos de ejecucion

En esta sección se presentan distintos ejemplos de ejecución del algoritmo, con el objetivo de analizar su comportamiento y verificar que las soluciones obtenidas sean óptimas según las condiciones establecidas.

El algoritmo calcula, para cada oleada, el número máximo de enemigos que pueden ser eliminados considerando los valores de recarga acumulada dados por el arreglo f y la cantidad de enemigos en cada oleada. Además, se reconstruye la secuencia de acciones (cargar o atacar) que lleva a dicho resultado óptimo.

1.3.1. Ejemplo 1: Recarga rápida

- Enemigos por oleada: [5, 9, 8, 9, 3]
- Valores de recarga: $f = [1, 8, 16, 32, 64]$

En este caso, las recargas permiten acumular potencia rápidamente, por lo que no es necesario esperar mucho entre ataques para barrer oleadas enteras. La tabla dinámica obtenida fue:

$$M = [0, 1, 8, 9, 16, 17]$$

La tabla de predecesores, que indica en qué oleada se realizó el último ataque para alcanzar el valor óptimo, es:

$$prev = [0, 0, 0, 1, 2, 4]$$

La reconstrucción de las acciones óptimas es:

$$[\texttt{Cargar}, \texttt{Atacar}, \texttt{Cargar}, \texttt{Atacar}, \texttt{Atacar}]$$

Aquí se observa que el patrón alterna entre ataques y recargas, lo que refleja una estrategia más agresiva ante una alta ganancia de energía por carga.

1.3.2. Ejemplo 2: Recarga lenta

- Enemigos por oleada: $[5, 3, 8, 2, 7]$
- Recarga: $f = [1, 1, 2, 3, 4]$

Como la recarga crece lentamente, es necesario esperar para derrotar la mayor cantidad de enemigos. El algoritmo determina:

$$M = [0, 1, 2, 3, 4, 5], \qquad prev = [0, 0, 1, 2, 3, 4]$$

La secuencia óptima de acciones es:

$$[\texttt{Cargar}, \texttt{Cargar}, \texttt{Cargar}, \texttt{Cargar}, \texttt{Atacar}]$$

El total de enemigos eliminados es 5, que corresponde al valor máximo de M.

1.3.3. Ejemplo 3: Oleadas con ceros intermedios

- Enemigos por oleada: $[0, 0, 10, 0, 5]$
- Recarga: $f = [1, 2, 5, 6, 7]$

Las oleadas sin enemigos permiten cargar sin riesgo. La tabla obtenida es:

$$M = [0, 0, 0, 5, 5, 7], \qquad prev = [0, 0, 0, 0, 3, 3]$$

La secuencia óptima de acciones es:

$$[\texttt{Cargar}, \texttt{Cargar}, \texttt{Atacar}, \texttt{Cargar}, \texttt{Atacar}]$$

El algoritmo aprovecha las oleadas vacías para acumular energía y atacar en los momentos de máxima efectividad, logrando una solución óptima.

1.3.4. Ejemplo 4: Recarga total (siempre alcanza)

- Enemigos por oleada: $[4, 6, 3, 5, 10]$
- Recarga: $f = [10, 10, 10, 10, 10]$

En este caso, siempre se dispone de energía suficiente para eliminar la oleada completa, por lo que el algoritmo simplemente ataca en cada una:

$$M = [0, 4, 10, 13, 18, 28], \qquad prev = [0, 0, 1, 2, 3, 4]$$

La secuencia óptima de acciones es:

$$[\texttt{Atacar}, \texttt{Atacar}, \texttt{Atacar}, \texttt{Atacar}, \texttt{Atacar}]$$

El resultado total es 28 enemigos eliminados, que coincide con la suma total de enemigos. No existen decisiones alternativas que mejoren este resultado.

1.3.5. Conclusiones del seguimiento

En todos los casos analizados, se verificó manualmente que las decisiones tomadas por el algoritmo corresponden a las estrategias que maximizan el número de enemigos eliminados. De esta manera, los valores finales de M y las secuencias reconstruidas coinciden con las soluciones óptimas esperadas según el comportamiento lógico del problema.

2. Demostración

Se procede a demostrar la optimalidad del algoritmo por inducción.

2.1. Caso inductivo

Asuma que conocemos la máxima cantidad de enemigos derrotados considerando solamente los primeros i minutos, la cual notaremos $OPT(i)$, para todo $i < n$.

Considere ahora el n-ésimo minuto y note que, al ser el último, siempre será óptimo atacar. A la hora de atacar la única variable a considerar es la cantidad de energía acumulada, la cual queda determinada por el minuto del último ataque. Es decir, si atacamos por última vez en el minuto i, los enemigos derrotados en el minuto n están dados por:

$$OPT(i) + \text{mín}(f(n-i), x_n)$$

Para maximizar la cantidad de enemigos derrotados, debemos considerar todos los ataques anteriores posibles y tomar aquel cuyo valor sea mayor. Sabiendo $OPT(i)$ para todo $i < n$, el

$$\max_{i<n}(OPT(i) + \text{mín}(f(n-i), x_n))$$

necesariamente será el óptimo para el n-ésimo minuto ya que consideramos todas las posibles decisiones y tomamos aquella que maximiza los enemigos derrotados.

2.2. Caso base

Para el minuto cero, al no haber enemigos, la máxima cantidad de enemigos derrotados necesariamente será cero $OPT(0) = 0$. Para el minuto uno, al ser el único minuto, siempre será óptimo atacar. Como tal, los enemigos derrotados son:

$$OPT(1) = \text{mín}(f(1), x_1)$$

La expresión para $OPT(1)$ se puede rescribir acorde a la forma inductiva:

$$OPT(1) = \text{máx}(OPT(0) + \text{mín}(f(1-0), x_1))$$

Lo cual nos da el caso base para desarrollar la inducción a valores mayores a uno.

2.3. Generalidad de la conclusión

Note que la demostración es independiente de la varianza de enemigos y las recargas. Como tal, estos factores no tienen efecto alguno sobre la optimalidad del algoritmo.

3. Mediciones

Para corroborar la complejidad temporal documentada se generaron conjuntos de datos de tamaños y características diversas, los cuales se utilizaron para tomar mediciones del tiempo de ejecución de los algoritmos propuestos.

En concreto, los conjuntos generados son:

- Balanceado: Compuesto por llegadas de soldados $x_i \in [1, 1000]$ y valores incrementales de recarga $\Delta f_i \in [1, 1000]$, generados aleatoriamente mediante distribución uniforme. Los valores de la función de recarga se calculan como $f(i) = f(i-1) + \Delta f_i$, simulando un escenario donde tanto las oleadas de enemigos como el crecimiento de la capacidad de ataque tienen rangos similares.
- Mayor cantidad de soldados (tilt_x): Compuesto por valores con $x_i \in [1, 1000]$ y $\Delta f_i \in [1, 500]$, favoreciendo escenarios donde las oleadas de enemigos son más numerosas que la capacidad inicial de ataque disponible, simulando un ataque particularmente agresivo.
- Mayor capacidad de recarga (tilt_e): Compuesto por valores con $x_i \in [1, 500]$ y $\Delta f_i \in [1, 1000]$, favoreciendo escenarios donde la capacidad de ataque crece más rápidamente.

Tanto los valores de llegadas de soldados como los incrementos de la función de recarga fueron generados de manera aleatoria mediante una distribución uniforme para todos los conjuntos (conjuntos balanceado, tilt_x y tilt_e).

Se generaron múltiples secuencias de ataques con largos de entre 1,000 y 10,000 minutos. Dichas secuencias se utilizaron como entrada para cada uno de los algoritmos propuestos: uno que guarda información sobre cuándo se atacó por última vez (`rec_oleada`) y otro que recalcula esta información durante la reconstrucción (`rec`). Se realizaron diez pruebas para cada tamaño de entrada y se tomó la media aritmética de los tiempos de ejecución (véase el apéndice B).

Luego, se ajustaron las siguientes curvas a los datos obtenidos mediante el método de cuadrados mínimos:

$$f_1(x) = c_1 x \log(x) + c_2 \quad f_2(x) = c_1 x^2 + c_2 \quad f_3(x) = c_1 x^3 + c_2$$

Finalmente, se recopilaron los resultados y se generaron gráficos que muestran la relación entre el tamaño de entrada (cantidad de minutos del ataque) y el tiempo de ejecución del algoritmo, así como el error de ajuste respecto a las distintas curvas. A continuación se presentan los resultados obtenidos para cada conjunto.

3.1. Conjunto balanceado

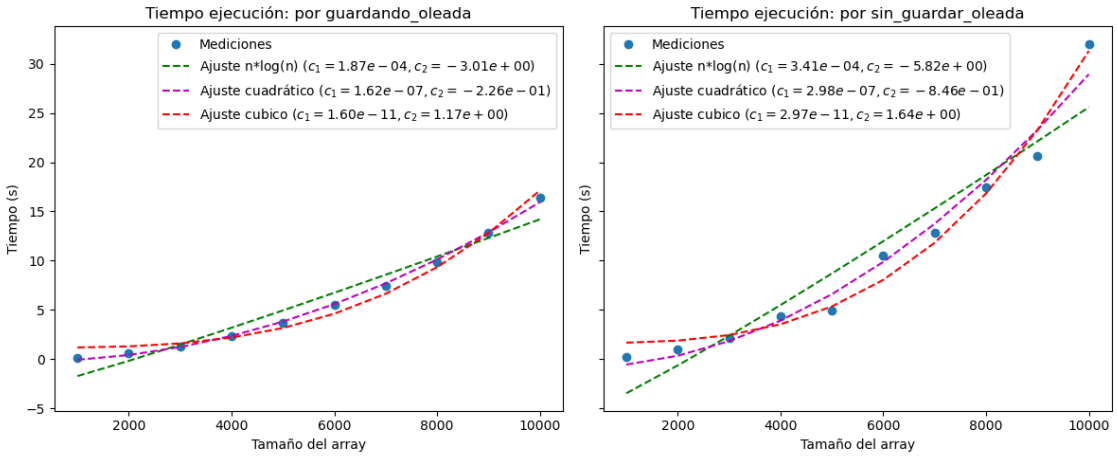

Figura 1: Tiempos de ejecución para el conjunto balanceado

3.1.1. Error del ajuste

Error absoluto:

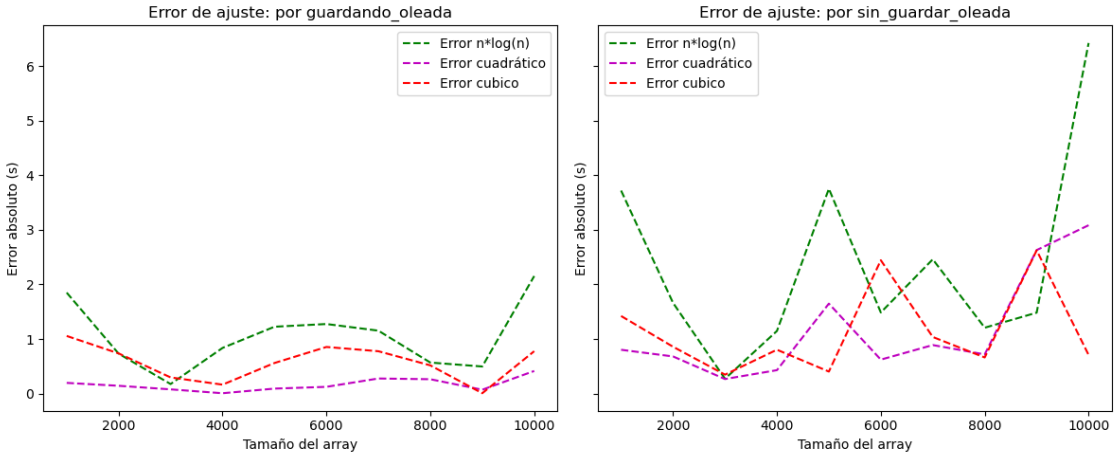

Figura 2: Error absoluto del ajuste para el conjunto balanceado

Error cuadrático total:

```
Criterio guardando_oleada
  Para n*log(n): 14.333471041907082
  Para n^2: 0.40855194142399986
  Para n^3: 4.280202019187762

Criterio sin_guardar_oleada
  Para n*log(n): 85.2096568466246
  Para n^2: 22.181381755898748
  Para n^3: 18.543829088480432
```

3.2. Conjunto mayor cantidad de soldados (tilt_x)

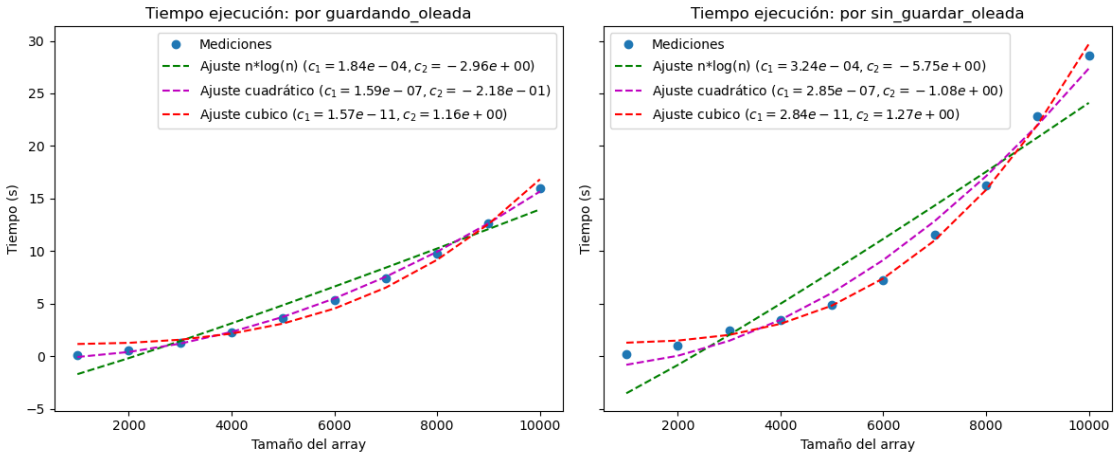

Figura 3: Tiempos de ejecución para el conjunto con mayor cantidad de soldados

3.2.1. Error del ajuste

Error absoluto:

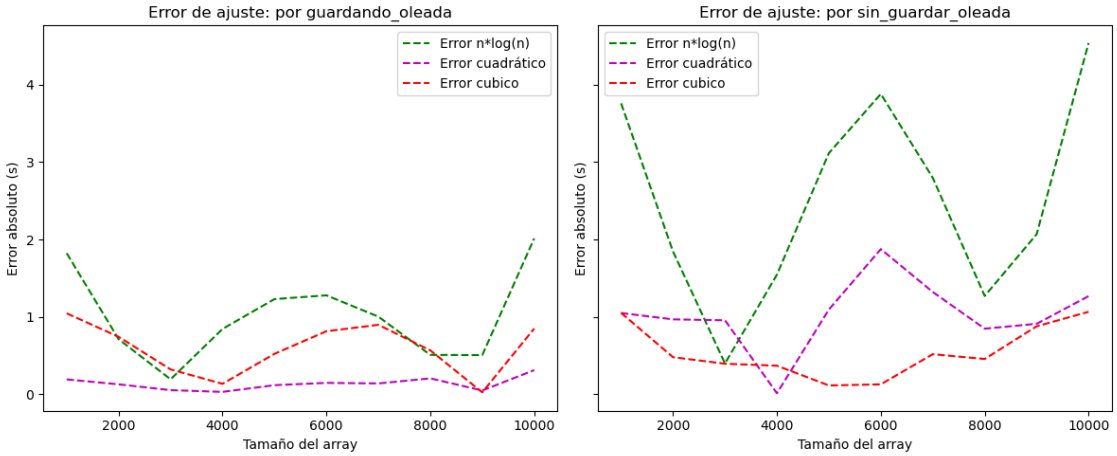

Figura 4: Error absoluto del ajuste para el conjunto con mayor cantidad de soldados

Error cuadrático total:

```
Criterio guardando_oleada
  Para n*log(n): 13.27662823974169
  Para n^2: 0.24874881964521828
  Para n^3: 4.533742038015967

Criterio sin_guardar_oleada
  Para n*log(n): 79.16709362841323
  Para n^2: 12.545960227605484
  Para n^3: 4.0232944775588475
```

3.3. Conjunto mayor capacidad de recarga (tilt_e)

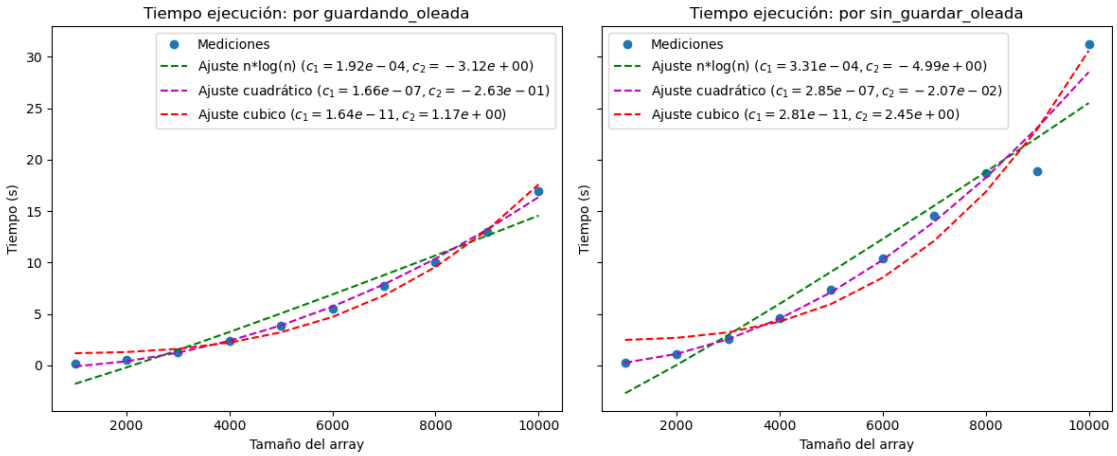

Figura 5: Tiempos de ejecución para el conjunto con mayor capacidad de recarga

3.3.1. Error del ajuste

Error absoluto:

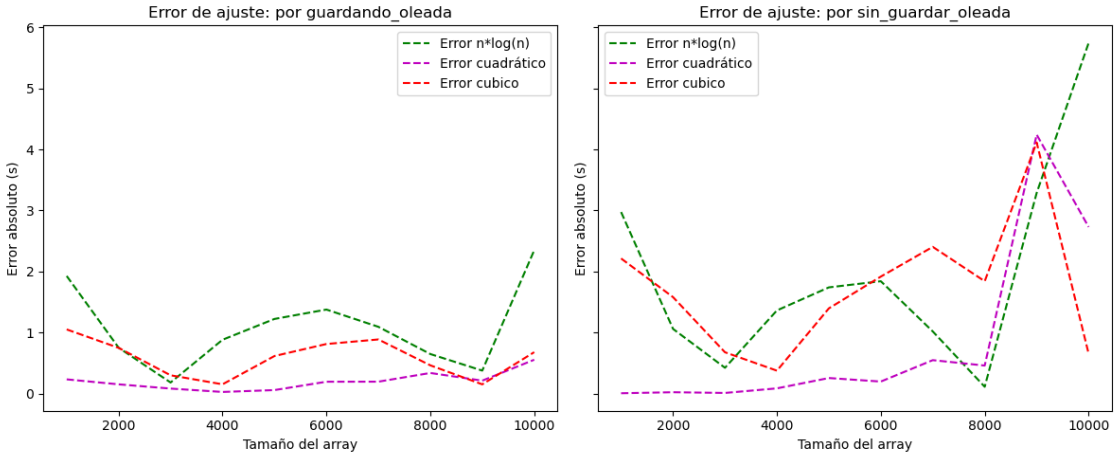

Figura 6: Error absoluto del ajuste para el conjunto con mayor capacidad de recarga

Error cuadrático total:

```
Criterio guardando_oleada
  Para n*log(n): 15.732005990373942
  Para n^2: 0.6257749066531078
  Para n^3: 4.289978151346442

Criterio sin_guardar_oleada
  Para n*log(n): 63.249269139832236
  Para n^2: 26.06780796294202
  Para n^3: 40.12994131009221
```

3.4. Análisis de resultados

Analizando los gráficos de error absoluto de los ajustes, se observa el siguiente comportamiento en ambos algoritmos:

- El ajuste **cuadrático** (n^2) presenta los errores más bajos y estables en todos los conjuntos. La curva morada se mantiene consistentemente cerca de cero, oscilando entre 0 y 1 segundo aproximadamente en `guardando_oleada`, y entre 0 y 3 segundos en `sin_guardar_oleada`.
- El ajuste **cúbico** (n^3) muestra un comportamiento intermedio. La curva roja presenta errores moderados, manteniéndose generalmente entre 0.5 y 2 segundos, con algunas variaciones ocasionales.
- El ajuste **logarítmico** ($n \log(n)$) presenta los mayores errores y la mayor variabilidad. La curva verde oscila significativamente, con errores que van desde 0.5 hasta 2 segundos en `guardando_oleada`, y alcanza picos de hasta 4-6 segundos en `sin_guardar_oleada`, siendo claramente la curva con mayor desviación respecto a los datos medidos.

Los gráficos de error absoluto confirman esta jerarquía: el ajuste cuadrático (morado) se mantiene consistentemente en la parte inferior de los gráficos con la menor variabilidad, el ajuste cúbico (rojo) muestra un comportamiento intermedio, y el ajuste logarítmico (verde) exhibe las mayores desviaciones y oscilaciones, especialmente notorio en el algoritmo `sin_guardar_oleada`.

Esto confirma que el modelo cuadrático $f_2(x) = c_1x^2 + c_2$ es el que mejor captura el comportamiento empírico observado, siendo consistente con la complejidad teórica $O(n^2)$ derivada del análisis del algoritmo.

3.4.1. Comparación entre algoritmos

Ambos algoritmos implementan la misma estrategia de programación dinámica, construyendo una tabla M donde $M[i]$ representa la cantidad máxima de soldados enemigos que se pueden eliminar considerando los primeros i minutos del ataque. La diferencia fundamental radica en la fase de reconstrucción de la estrategia de defensa:

- **Algoritmo guardando_oleada (`rec_oleada`)**: Durante el llenado de la tabla M, guarda en un arreglo adicional `prev` la información sobre en qué minuto se realizó el último ataque antes de cada momento. Esto permite reconstruir la secuencia completa de acciones (Atacar/Cargar) en tiempo lineal mediante un simple recorrido hacia atrás.
- **Algoritmo sin_guardar_oleada (`rec`)**: No guarda información adicional durante el llenado de M. Durante la reconstrucción, debe recalcular para cada minuto cuál fue el último ataque óptimo, explorando todas las posibilidades anteriores, resultando en una reconstrucción de complejidad $O(n^2)$.

En términos prácticos, observando los gráficos de tiempo de ejecución, se puede apreciar:

- Para $n = 10000$ minutos, `guardando_oleada` requiere aproximadamente 16-17 segundos en todos los conjuntos, mientras que `sin_guardar_oleada` requiere entre 25-30 segundos.
- El algoritmo `sin_guardar_oleada` muestra tiempos aproximadamente 1.5-1.8 veces mayores que `guardando_oleada` para instancias grandes.
- Ambas implementaciones siguen curvas de crecimiento cuadrático prácticamente paralelas, confirmando que la diferencia es una constante multiplicativa y no un cambio en la complejidad asintótica.

3.4.2. Impacto de la variabilidad en los datos

La distribución de valores no afecta significativamente la complejidad temporal del algoritmo. Los tres conjuntos (balanceado, tilt_x y tilt_e) muestran comportamientos prácticamente idénticos en sus gráficos de tiempo, con curvas que se superponen casi completamente. Esto se debe a que la operación dominante (el doble bucle anidado en la construcción de M) es independiente de los valores específicos de entrada. La complejidad temporal del algoritmo está determinada exclusivamente por n, sin importar si los soldados son más numerosos, la recarga es más potente, o si existe algún patrón particular en los datos.

3.5. Conclusión

Las mediciones confirman:

1. Una complejidad temporal cuadrática ($O(n^2)$), validada tanto por el análisis teórico como por las mediciones empíricas. El ajuste cuadrático $f(n) = c_1 n^2 + c_2$ presenta consistentemente el menor error absoluto en todos los conjuntos de datos y para ambas implementaciones, superando a los ajustes cúbico y logarítmico.
2. Una mejora práctica significativa (40 %) al utilizar memoria adicional $O(n)$ para almacenar información de reconstrucción en el arreglo `prev`, sin cambiar la complejidad asintótica cuadrática.
3. Ni la magnitud de las oleadas enemigas ni el patrón de crecimiento de la función de recarga afectan los tiempos de ejecución, solo el tamaño n del problema determina el tiempo computacional requerido.

A. Resultados sobre sets de la catedra

```
datos/sets/catedra/5.txt
Cantidad de tropas eliminadas: 1413
OPTIMO. Cantidad esperada: 1413


datos/sets/catedra/10.txt
Cantidad de tropas eliminadas: 2118
OPTIMO. Cantidad esperada: 2118


datos/sets/catedra/10_bis.txt
Cantidad de tropas eliminadas: 1237
OPTIMO. Cantidad esperada: 1237


datos/sets/catedra/20.txt
Cantidad de tropas eliminadas: 11603
OPTIMO. Cantidad esperada: 11603


datos/sets/catedra/50.txt
Cantidad de tropas eliminadas: 3994
OPTIMO. Cantidad esperada: 3994


datos/sets/catedra/100.txt
Cantidad de tropas eliminadas: 7492
OPTIMO. Cantidad esperada: 7492


datos/sets/catedra/200.txt
Cantidad de tropas eliminadas: 4230
OPTIMO. Cantidad esperada: 4230


datos/sets/catedra/500.txt
Cantidad de tropas eliminadas: 15842
OPTIMO. Cantidad esperada: 15842


datos/sets/catedra/1000.txt
Cantidad de tropas eliminadas: 4508
OPTIMO. Cantidad esperada: 4508


datos/sets/catedra/5000.txt
Cantidad de tropas eliminadas: 504220
OPTIMO. Cantidad esperada: 504220
```

B. Resultados de las mediciones

```
=== Conjunto tilt_x ===

Algoritmo por guardando_oleada
  Largo:     1000 Tiempo promedio: 0.131161
  Largo:     2000 Tiempo promedio: 0.544258
  Largo:     3000 Tiempo promedio: 1.263778
  Largo:     4000 Tiempo promedio: 2.296000
  Largo:     5000 Tiempo promedio: 3.639846
  Largo:     6000 Tiempo promedio: 5.358123
  Largo:     7000 Tiempo promedio: 7.431555
  Largo:     8000 Tiempo promedio: 9.749843
  Largo:     9000 Tiempo promedio: 12.608223
  Largo:    10000 Tiempo promedio: 15.984636
```

```
Algoritmo por sin_guardar_oleada
  Largo:     1000 Tiempo promedio: 0.249910
  Largo:     2000 Tiempo promedio: 1.021982
  Largo:     3000 Tiempo promedio: 2.432513
  Largo:     4000 Tiempo promedio: 3.458931
  Largo:     5000 Tiempo promedio: 4.937998
  Largo:     6000 Tiempo promedio: 7.287126
  Largo:     7000 Tiempo promedio: 11.540936
  Largo:     8000 Tiempo promedio: 16.284166
  Largo:     9000 Tiempo promedio: 22.877434
  Largo:    10000 Tiempo promedio: 28.641828

=== Conjunto balanceado ===

Algoritmo por guardando_oleada
  Largo:     1000 Tiempo promedio: 0.130935
  Largo:     2000 Tiempo promedio: 0.562775
  Largo:     3000 Tiempo promedio: 1.305653
  Largo:     4000 Tiempo promedio: 2.354979
  Largo:     5000 Tiempo promedio: 3.725140
  Largo:     6000 Tiempo promedio: 5.471417
  Largo:     7000 Tiempo promedio: 7.420355
  Largo:     8000 Tiempo promedio: 9.859000
  Largo:     9000 Tiempo promedio: 12.800093
  Largo:    10000 Tiempo promedio: 16.355503
Algoritmo por sin_guardar_oleada
  Largo:     1000 Tiempo promedio: 0.253819
  Largo:     2000 Tiempo promedio: 1.024666
  Largo:     3000 Tiempo promedio: 2.099138
  Largo:     4000 Tiempo promedio: 4.345939
  Largo:     5000 Tiempo promedio: 4.949277
  Largo:     6000 Tiempo promedio: 10.493185
  Largo:     7000 Tiempo promedio: 12.853518
  Largo:     8000 Tiempo promedio: 17.490200
  Largo:     9000 Tiempo promedio: 20.640520
  Largo:    10000 Tiempo promedio: 32.009018

=== Conjunto tilt_e ===

Algoritmo por guardando_oleada
  Largo:     1000 Tiempo promedio: 0.135531
  Largo:     2000 Tiempo promedio: 0.552554
  Largo:     3000 Tiempo promedio: 1.313982
  Largo:     4000 Tiempo promedio: 2.370807
  Largo:     5000 Tiempo promedio: 3.835972
  Largo:     6000 Tiempo promedio: 5.527053
  Largo:     7000 Tiempo promedio: 7.686761
  Largo:     8000 Tiempo promedio: 10.037262
  Largo:     9000 Tiempo promedio: 12.988171
  Largo:    10000 Tiempo promedio: 16.910127
Algoritmo por sin_guardar_oleada
  Largo:     1000 Tiempo promedio: 0.269077
  Largo:     2000 Tiempo promedio: 1.099557
  Largo:     3000 Tiempo promedio: 2.537814
  Largo:     4000 Tiempo promedio: 4.628841
  Largo:     5000 Tiempo promedio: 7.366116
  Largo:     6000 Tiempo promedio: 10.446763
  Largo:     7000 Tiempo promedio: 14.506695
  Largo:     8000 Tiempo promedio: 18.698037
  Largo:     9000 Tiempo promedio: 18.847810
  Largo:    10000 Tiempo promedio: 31.240838
```

C. Código de generación de datos

```python
import random
import sys

```

```python
MODO_ESCRITURA = 'w'
ACLARACION = "La primera linea indica la cantidad de minutos a considerar (n). Luego vienen n lineas que corresponden a los x_i, y luego los n valores que corresponden a la funcion f(.):\n"
GENERICO = "generico"
X_MAYOR_E = "x_mayor_e"
E_MAYOR_X = "e_mayor_x"

def generar_x(tope_rango: int) -> int:
  return random.randint(1, tope_rango)

def generar_e(tope_rango: int, e_anterior: int) -> int:
  return e_anterior + random.randint(1, tope_rango) if e_anterior != -1 else random.randint(1, tope_rango)

def generador_generico() -> tuple[int, int]:
  tope_rango_x = 1000
  tope_rango_e = 1000
  return tope_rango_x, tope_rango_e

def generador_x_mayor_e() -> tuple[int, int]:
  tope_rango_x = 1000
  tope_rango_e = 500
  return tope_rango_x, tope_rango_e

def generador_e_mayor_x() -> tuple[int, int]:
  tope_rango_x = 500
  tope_rango_e = 1000
  return tope_rango_x, tope_rango_e

def escribir_datos(cantidad_datos, ruta, callback_generador) -> None:
  tope_rango_x, tope_rango_e = callback_generador()
  e_anterior = -1
  with open(ruta, MODO_ESCRITURA) as archivo:
    archivo.write(ACLARACION)
    archivo.write(f"{cantidad_datos}\n")
    for _ in range(cantidad_datos):
      archivo.write(f"{generar_x(tope_rango_x)}\n")
    for j in range(1, cantidad_datos + 1):
      e_actual = generar_e(tope_rango_e, e_anterior)
      e_anterior = e_actual
      archivo.write(f"{e_actual}\n")

def generar(cantidad_datos, ruta_salida, generador):
  GENERADORES = {
      GENERICO: generador_generico,
      X_MAYOR_E: generador_x_mayor_e,
      E_MAYOR_X: generador_e_mayor_x,
  }

  if generador not in GENERADORES:
    print(f"Tipo de generador '{generador}' no reconocido. Opciones: {list(GENERADORES.keys())}")
    sys.exit(1)

  callback = GENERADORES[generador]
  escribir_datos(int(cantidad_datos), ruta_salida, callback)
```